CLASS 12 — COMPUTER SCIENCE (C++)

Bihar Board (BSEB)

CHAPTER 4

Boolean Algebra

Complete Study Notes — Simple English

Number System • Logic Gates • Boolean Laws • PYQs • Exam Tips

Subject	Computer Science (C++)
Chapter	4 — Boolean Algebra
Board	Bihar School Examination Board (BSEB)
Language	Simple English
Level	Beginner to Advanced — Step by Step

How to Use These Notes

Step 1	Read each topic from top to bottom — they are arranged in order.
Step 2	Read the definition box first. It tells you 'What' the topic is.
Step 3	Look at the worked examples carefully. Try them yourself.
Step 4	Check the Truth Tables. Draw them on paper once.
Step 5	Solve the PYQ section before your exam.
Step 6	Use the Quick Formula Card on the last page before the exam.

Table of Contents

PART A — NUMBER SYSTEM

Topic 1: What is a Number System?

What is Number System?

A Number System is a way of writing and counting numbers using certain symbols (called digits). Just like we use 0–9 in our daily life, a computer uses only 0 and 1. That is why we need to learn different number systems.

Why does a computer use only 0 and 1?

A computer is made of tiny switches. Each switch can be either OFF (= 0) or ON (= 1). So a computer stores and processes everything — text, images, video — using only 0s and 1s.

■ Exam Tip — 4 Types to Remember

There are 4 types of number systems you must know for the exam: Decimal (base 10), Binary (base 2), Octal (base 8), Hexadecimal (base 16).

Topic 2: Types of Number Systems

Name	Base	Digits Used	Real-life Use
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	Used by humans in daily life
Binary	2	0, 1	Used inside every computer
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	Short form of Binary
Hexadecimal	16	0-9 and A, B, C, D, E, F	Used in memory addresses

Decimal System (Base 10)

This is the number system we use every day. It has 10 digits: 0 to 9. The value of each digit depends on its position (ones, tens, hundreds...).

Example: $425 = 4 \times 10^2 + 2 \times 10^1 + 5 \times 10^0 = 400 + 20 + 5 = 425$

Binary System (Base 2)

This is the language of computers. It has only 2 digits: 0 and 1. Each digit is called a Bit. 8 bits = 1 Byte. The value of each position is a power of 2.

Example: $(1011)_2$ = 1x8 + 0x4 + 1x2 + 1x1 = 8+0+2+1 = $(11)_{10}$

Octal System (Base 8)

This system has 8 digits: 0 to 7. It is useful because 3 binary digits make 1 octal digit. Easy to convert from Binary.

Example: $(17)_8$ = 1x8 + 7x1 = 8 + 7 = $(15)_{10}$

Hexadecimal System (Base 16)

This system has 16 symbols: digits 0-9 and letters A(=10), B(=11), C(=12), D(=13), E(=14), F(=15). 4 binary digits = 1 hex digit. Used in memory addresses and color codes.

Example: $(1F)_{16}$ = 1x16 + 15x1 = 16 + 15 = $(31)_{10}$

Hexadecimal Quick Reference Chart:

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Topic 3: Number Conversion — Step by Step

Converting a number from one system to another is very important for the exam. Follow the steps given below carefully.

Decimal → Binary

Divide the number by 2 again and again. Write the remainders from BOTTOM to TOP.

```
Convert (13)10 to Binary:

13 / 2 = 6 remainder 1

6 / 2 = 3 remainder 0

3 / 2 = 1 remainder 1

1 / 2 = 0 remainder 1

Read remainders bottom to top: 1 1 0 1

Answer: (13)10 = (1101)2
```

Binary → Decimal

Multiply each bit by its position value (power of 2) and add them all up.

Convert (1010)2 to Decimal:

$= 1x2^3 + 0x2^2 + 1x2^1 + 0x2^0$

= 8 + 0 + 2 + 0

Answer: (1010)2 = (10)10

Decimal → Octal

Divide the number by 8 again and again. Write the remainders from BOTTOM to TOP.

```
Convert (65)10 to Octal:

65 / 8 = 8 remainder 1

8 / 8 = 1 remainder 0

1 / 8 = 0 remainder 1

Read remainders bottom to top: 1 0 1

Answer: (65)10 = (101)8
```

Binary → Octal

Group the binary digits into sets of 3 (from RIGHT to LEFT). Then convert each group of 3 bits to its octal digit.

```
Convert (110101)2 to Octal:

Group: 110 101

Value: 6 5

Answer: (110101)2 = (65)8
```

Decimal → Hexadecimal

Divide the number by 16. If remainder is 10-15, write A-F.

```
Convert (255)10 to Hexadecimal:

255 / 16 = 15 remainder 15 (= F)

15 / 16 = 0 remainder 15 (= F)

Read remainders bottom to top: F F

Answer: (255)10 = (FF)16
```

Binary → Hexadecimal

Group the binary digits into sets of 4 (from RIGHT to LEFT). Then convert each group of 4 bits to its hex digit (0-9 or A-F).

```
Convert (11111111)2 to Hexadecimal:

Group: 1111 1111

Value: F F

Answer: (11111111)2 = (FF)16
```

■ Exam Tip — Conversion Shortcuts

Decimal to Binary = divide by 2 | Decimal to Octal = divide by 8 | Decimal to Hex = divide by 16 Binary to Octal = make groups of 3 | Binary to Hex = make groups of 4 Always read remainders from BOTTOM to TOP.

PART B — BOOLEAN ALGEBRA BASICS

Topic 4: What is Boolean Algebra?

What is Boolean Algebra?

Boolean Algebra is a branch of mathematics that works with only two values: TRUE (= 1) and FALSE (= 0). It was invented by the mathematician George Boole in the 19th century. Every computer, mobile phone, and digital device uses Boolean Algebra inside.

Why is it important?

- ✔ All data inside a computer is stored as 0s and 1s.
- ✔ Logic gates and circuits in a computer are based on Boolean Algebra.
- ✔ The if-else conditions in C++ programs use Boolean logic.
- ✔ BSEB exam has 10–15 marks from this chapter alone.

Topic 5: Boolean Values and Variables

Value	Symbol	Real-world Example
TRUE	1 (High)	Switch is ON / Bulb is glowing
FALSE	0 (Low)	Switch is OFF / Bulb is not glowing

What is a Boolean Variable?

A Boolean variable is a variable that can store only one of two values: 0 or 1. We use capital letters like A, B, C, X, Y to represent them.

```
bool A = true; // A = 1
bool B = false; // B = 0
bool result = A && B; // AND operation
```

Topic 6: The Three Basic Operations

There are 3 basic operations in Boolean Algebra. Everything else is built from these three.

AND (A.B)

AND gives output 1 only when BOTH inputs are 1. If even one input is 0, output is 0. In C++, AND is written as &&

Trick to Remember: Think of AND like multiplication: 1 x 1 = 1, but 1 x 0 = 0.

OR (A+B)

OR gives output 1 when AT LEAST ONE input is 1. Output is 0 only when BOTH inputs are 0. In C++, OR is written as ||

Trick to Remember: Think of OR like addition: 0+0=0, but 0+1=1 and 1+1=1.

NOT (A')

NOT flips the input. If input is 1, output is 0. If input is 0, output is 1. It has only ONE input. In C++, NOT is written as !A Other symbols: A' or A-bar (A with a line on top).

Trick to Remember: Think of NOT as 'opposite' — whatever you give, it returns the reverse.

C++ code showing all three operations:

```
#include
using namespace std;
int main() {
bool A = 1, B = 0;
cout << (A && B) << endl; // AND Output: 0
cout << (A || B) << endl; // OR Output: 1
cout << (!A) << endl; // NOT Output: 0
return 0;
}
```

Topic 7: Truth Tables — What are they?

What is Truth Table?

A Truth Table is a table that shows the OUTPUT of a Boolean expression for EVERY POSSIBLE combination of inputs. Rule: if there are n inputs, there will be 2(n) rows in the truth table. Example: 2 inputs = 4 rows, 3 inputs = 8 rows.

AND (A.B)

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Note: Output is 1 ONLY when both A=1 and B=1.

OR (A+B)

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

Note: Output is 0 ONLY when both A=0 and B=0.

NOT (A')

A	NOT A
0	1
1	0

Note: Only 2 rows because there is only 1 input.

NAND ((A.B)')

A	B	NAND (A.B)'
0	0	1
0	1	1

1	0	1
1	1	0

Note: NAND = opposite of AND. It is a Universal Gate.

NOR ((A+B)')

A	B	NOR (A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

Note: NOR = opposite of OR. It is also a Universal Gate.

XOR (A XOR B)

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Note: Output is 1 when inputs are DIFFERENT. Output is 0 when inputs are SAME.

XNOR ((A XOR B)')

A	B	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Note: XNOR = opposite of XOR. Output is 1 when inputs are SAME.

■ Exam Tip — How to draw Truth Tables

Always write inputs in this order: 0,0 then 0,1 then 1,0 then 1,1. First write all input columns, then intermediate columns, then the final output column. n inputs = 2(n) rows.

PART C — LOGIC GATES

Topic 8: What is a Logic Gate?

What is Logic Gate?

A Logic Gate is an electronic circuit that performs a Boolean operation. It takes one or more inputs (only 0 or 1) and gives one output (0 or 1). Logic gates are the building blocks of all computers and digital devices. They are made from tiny transistors inside a chip.

Topic 9: All Logic Gates — Explained Simply

AND Gate — Symbol: A . B

Output is 1 ONLY when BOTH inputs are 1.

Easy Trick: Think: BOTH must agree.

OR Gate — Symbol: A + B

Output is 1 when AT LEAST ONE input is 1.

Easy Trick: Think: EITHER one is enough.

NOT Gate — Symbol: A'

Flips the input. 0 becomes 1, and 1 becomes 0. Has only 1 input.

Easy Trick: Think: It is the opposite gate.

NAND Gate — Symbol: (A.B)'

NAND = NOT + AND. It is the opposite of AND. Output is 0 ONLY when BOTH inputs are 1. NAND is a Universal Gate — you can build any other gate using only NAND gates.

Easy Trick: NAND = AND flipped. ★ Universal Gate

NOR Gate — Symbol: (A+B)'

NOR = NOT + OR. It is the opposite of OR. Output is 1 ONLY when BOTH inputs are 0. NOR is also a Universal Gate.

Easy Trick: NOR = OR flipped. ★ Universal Gate

XOR Gate — Symbol: A XOR B

XOR = Exclusive OR. Output is 1 when inputs are DIFFERENT. If both inputs are same (both 0 or both 1), output is 0.

Easy Trick: Think: They must be different.

XNOR Gate — Symbol: (A XOR B)'

XNOR = NOT of XOR. Output is 1 when inputs are SAME. It is the opposite of XOR.

Easy Trick: Think: They must be same.

Topic 10: Universal Gates — NAND and NOR

What is Universal Gate?

A Universal Gate is a gate from which you can build ANY other gate (AND, OR, NOT, XOR — everything) using only that one type of gate. NAND and NOR are both Universal Gates. This is why they are the most important gates in digital electronics.

Building NOT from NAND

```
Connect both inputs of a NAND gate to the same wire (input A).

A NAND A = (A.A)' = A' (this is NOT)
```

Building NOT from NOR

```
Connect both inputs of a NOR gate to the same wire (input A).

A NOR A = (A+A)' = A' (this is NOT)
```

■ Very Important — Universal Gates

NAND and NOR are the two Universal Gates. Exam question: 'Which gate is called a Universal Gate?' — Answer: NAND and NOR both. This question comes every year!

PART D — BOOLEAN LAWS AND THEOREMS

Topic 11: 9 Important Boolean Laws

Boolean Laws are rules that help us simplify complex Boolean expressions. Learn each law with its formula and a simple example.

1. Identity Law

A number combined with a neutral element gives the same number back.

Type	Formula	Plain English
AND form	**A . 1 = A**	1 is neutral for AND — A AND 1 always gives A
OR form	**A + 0 = A**	0 is neutral for OR — A OR 0 always gives A

2. Null Law (Dominance Law)

When a 'dominating' value is used, the result is always that value — no matter what A is.

Type	Formula	Plain English
AND form	**A . 0 = 0**	0 dominates AND — anything AND 0 is always 0
OR form	**A + 1 = 1**	1 dominates OR — anything OR 1 is always 1

3. Idempotent Law

When you use a variable with itself, the result is the same variable.

Type	Formula	Plain English
AND form	**A . A = A**	A AND A = A (same switch pressed twice = once)
OR form	**A + A = A**	A OR A = A (same thing OR itself = itself)

4. Complement Law

A variable combined with its opposite always gives a fixed result.

Type	Formula	Plain English
AND form	**A . A' = 0**	A AND (NOT A) = 0 — they cancel each other out
OR form	**A + A' = 1**	A OR (NOT A) = 1 — one of them must be 1

5. Double Complement Law

Applying NOT twice brings you back to the original value.

Type	Formula	Plain English
Only form	**(A')' = A**	NOT(NOT A) = A — two negatives make a positive

6. Commutative Law

The order of variables does not matter.

Type	Formula	Plain English
AND form	**A . B = B . A**	A AND B = B AND A (like: 2x3 = 3x2)
OR form	**A + B = B + A**	A OR B = B OR A (like: 2+3 = 3+2)

7. Associative Law

The grouping of variables does not change the result.

Type	Formula	Plain English
AND form	**A.(B.C) = (A.B).C**	Brackets can be moved freely in AND
OR form	**A+(B+C) = (A+B)+C**	Brackets can be moved freely in OR

8. Distributive Law

AND can be distributed over OR (and vice versa), just like algebra.

Type	Formula	Plain English
AND over OR	**A.(B+C) = A.B + A.C**	Open the bracket — multiply A with each term
OR over AND	**A+(B.C) = (A+B).(A+C)**	OR distributes over AND

9. Absorption Law

A larger expression can be absorbed into a smaller one. Very useful in simplification!

Type	Formula	Plain English
Form 1	**A + A.B = A**	A absorbs A.B — the result is just A
Form 2	**A.(A+B) = A**	A absorbs (A+B) — the result is just A

Topic 12: De Morgan's Theorem (Very Important!)

De Morgan's Theorem is the most important theorem in Boolean Algebra. Almost every BSEB exam paper has at least one question on this theorem.

Theorem	Formula	Simple Meaning
First Theorem	**(A+B)' = A'.B'**	The complement of OR = AND of complements
Second Theorem	**(A.B)' = A'+B'**	The complement of AND = OR of complements

■ **Trick to Remember De Morgan's**

Easy way to remember De Morgan's Theorem: Step 1: Put a complement on each variable separately. Step 2: Change OR to AND (or AND to OR). Example: (A+B)' — put ' on A and B → A' and B' — change + to . → A'.B'

Proof of First Theorem (A+B)' = A'.B' — using Truth Table:

A	B	A+B	(A+B)'	A'	B'	A'.B'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Columns (A+B)' and A'.B' are identical — Theorem is PROVED! ✓

Topic 13: Simplification of Boolean Expressions

Simplification means making a complex Boolean expression shorter and simpler by applying the Boolean laws step by step. In each step, write which law you used — in brackets.

Example 1 — Easy: Simplify: A.A' + B

```
= 0 + B [Complement Law: A.A' = 0]
= B [Identity Law: 0+B = B]
Final Answer: B
```

Example 2 — Absorption: Simplify: A + A.B

```
= A.(1 + B) [Distributive Law]

= A.1 [Null Law: 1+B = 1]

= A [Identity Law]

Final Answer: A
```

Example 3 — Medium: Simplify: A.B + A.B'

```
= A.(B + B') [Distributive Law]

= A.1 [Complement Law: B+B' = 1]

= A [Identity Law]

Final Answer: A
```

Example 4 — De Morgan's: Simplify: (A'+B')'

```
= A.B [De Morgan's Second Theorem: (A'+B')' = A.B]

Final Answer: A.B
```

Example 5 — Hard: Simplify: A.B + A'.B + A.B'

```
= B(A+A') + A.B' [Distributive Law]

= B.1 + A.B' [Complement Law: A+A' = 1]

= B + A.B' [Identity Law]

= A + B [Absorption Law: B + A.B' = A+B]

Final Answer: A + B
```

■ Exam Tip — How to Write Simplification

How to write simplification in the exam: 1. Write the expression clearly. 2. In each step, write the law used inside [ ] brackets. 3. Underline the Final Answer. The examiner checks your steps — don't skip them!

PART E — EXAM PREPARATION

Topic 14: Previous Year Questions (PYQs) — BSEB

Q. What is a Number System? Name its four types. [2 marks]

Ans:

A Number System is a way of writing numbers using certain digits.

Four types: (1) Decimal — base 10 (2) Binary — base 2

(3) Octal — base 8 (4) Hexadecimal — base 16

Q. Convert (1010)2 to Decimal. [2 marks]

Ans:

= 1x2(3) + 0x2(2) + 1x2(1) + 0x2(0)

= 8 + 0 + 2 + 0

Answer: (1010)2 = (10)10

Q. Convert (25)10 to Binary. [2 marks]

Ans:

25/2=12 rem 1, 12/2=6 rem 0, 6/2=3 rem 0, 3/2=1 rem 1, 1/2=0 rem 1

Read remainders bottom to top: 1 1 0 0 1

Answer: (25)10 = (11001)2

Q. What is Boolean Algebra? Name two basic operations. [2 marks]

Ans:

Boolean Algebra is a system of mathematics that works with only 0 (False) and 1 (True).

Two basic operations: (1) AND operation (A.B) — Logical Multiplication

(2) OR operation (A+B) — Logical Addition

Q. State De Morgan's Theorem and verify with a truth table. [5 marks]

Ans:

First Theorem: (A+B)' = A'.B'

Second Theorem: (A.B)' = A'+B'

Verification: Draw the truth table with columns for both sides.

Show that (A+B)' and A'.B' columns are identical. (See Topic 12 for full table.)

Q. Simplify: F = A.B + A.B' [3 marks]

Ans:

F = A.B + A.B'

= A.(B + B') [Distributive Law]

= A.1 [Complement Law: B+B'=1]

= A [Identity Law]

Final Answer: F = A

Q. What is a Universal Gate? Name the Universal Gates. [2 marks]

Ans:

A Universal Gate is a gate from which any other Boolean function can be built.

Universal Gates: (1) NAND Gate (2) NOR Gate

Using only NAND gates, you can build AND, OR, NOT, XOR — any circuit.

Q. Write the truth table for XOR and XNOR gates. [4 marks]

Ans:

XOR (A XOR B): 0,0 -> 0 | 0,1 -> 1 | 1,0 -> 1 | 1,1 -> 0

XNOR (A XNOR B): 0,0 -> 1 | 0,1 -> 0 | 1,0 -> 0 | 1,1 -> 1

(See Topic 7 for complete truth tables with proper format.)

Q. Convert (FF)16 to Decimal. [2 marks]

Ans:

= F x 16(1) + F x 16(0)

= 15 x 16 + 15 x 1

= 240 + 15

Answer: (FF)16 = (255)10

Topic 15: Quick Formula Card — Read Before Exam!

■ Marks Info

This chapter carries 10–15 marks in the BSEB exam. Number System + Boolean Algebra together — prepare both well!

Must-Know Points:

1. 4 number systems: Decimal(10), Binary(2), Octal(8), Hexadecimal(16).
2. Binary to Octal = group of 3 bits (right to left).
3. Binary to Hex = group of 4 bits (right to left).
4. Hex digits: A=10, B=11, C=12, D=13, E=14, F=15.
5. AND = multiply, OR = add, NOT = flip.
6. NAND and NOR are Universal Gates.
7. De Morgan's 1st: (A+B)' = A'.B'
8. De Morgan's 2nd: (A.B)' = A'+B'
9. Truth Table rows = 2(n) where n = number of inputs.
10. XOR: different inputs = 1, same inputs = 0.
11. XNOR: same inputs = 1, different inputs = 0.
12. Complement Law: A+A'=1 and A.A'=0 (very useful in simplification).
13. Identity Law: A+0=A and A.1=A
14. Double NOT Law: (A')' = A

Boolean Laws — Quick Reference Table:

Law Name	AND form	OR form
Identity	**A.1 = A**	**A+0 = A**
Null (Dominance)	**A.0 = 0**	**A+1 = 1**
Idempotent	**A.A = A**	**A+A = A**
Complement	**A.A' = 0**	**A+A' = 1**
Commutative	**A.B = B.A**	**A+B = B+A**
Absorption	**A.(A+B) = A**	**A+A.B = A**
De Morgan's 1st	**(A+B)'=A'.B'**	—
De Morgan's 2nd	**(A.B)'=A'+B'**	—
Double NOT	**(A')' = A**	—

All the Best for your Exam! You can do it! Practice one simplification every day and you will master this chapter. Bihar Board — Hard work always pays off!